

一、作品名称： 《十二周生存计划： 校园 RPG》

难度:☆☆☆☆

第一部分 功能实现（20 分）

代码测试游戏界面（10 分）

```
Shell x
Campus Life RPG: Path to Graduation
Survive 12 weeks and meet graduation requirements!

Enter your name: eunsong

Choose difficulty:
1. Easy
2. Normal
3. Hard
Enter difficulty: 2

Welcome, eunsong!
Difficulty selected: Normal
```

图 1.1 游戏开始界面（姓名输入 / 难度选择）

```
=====
[Week 2 Status]
=====
HP           [#####-----] 70/100
Knowledge    [#####-----] 45/100
Relationship [####-----] 20/100
Money        [#####-----] 50/100
Credits      [#-----] 1/12
=====
Current State: Study Needed: Your knowledge level is still low.
=====

Where do you want to go this week?
1. Library
2. Cafeteria
3. Club Room
4. Professor Office
5. Dorm
Enter your choice: |
```

图 2.1 游戏进行界面（Progress Bar 显示）

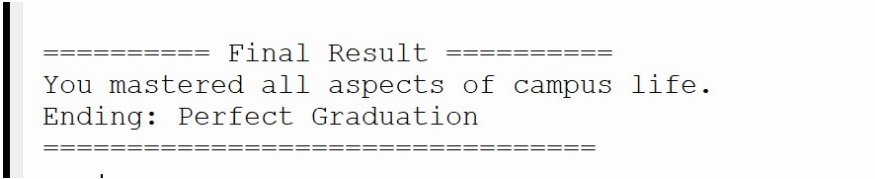


图 3 游戏结局界面

本游戏采用控制台文本界面，通过菜单选项与玩家进行交互。玩家输入姓名后选择游戏难度，系统根据难度设置不同初始属性，提高游戏挑战性。游戏运行过程中，系统实时显示周数、生命值、知识值、人际关系、金钱和学分等状态信息。程序使用文本进度条对属性进行可视化展示，方便玩家观察状态变化。程序支持数字与英文输入，并加入输入标准化与输入校验机制，可处理大小写、空输入和非法输入问题，提高程序稳定性与交互体验。游戏根据玩家最终的学分、知识值、人际关系和生命值判断不同结局，包括普通毕业、完美毕业、学业警告和毕业失败等结果。

设计思路游戏逻辑（10 分）：

设计思路：

本游戏以“十二周校园生存”为主题,通过状态管理系统模拟大学生活中的学习、社交、休息与经济管理。玩家需要在有限周数内合理安排行动，以满足毕业条件。程序采用模块化设计，将输入处理、状态显示、地点选择、事件执行和结局判断分别封装为独立函数，使程序结构更加清晰，便于维护与扩展。

游戏逻辑：

程序使用循环结构模拟十二周校园生活。每一周开始时，系统显示玩家当前状态，玩家随后选择行动地点。不同地点会触发不同随机事件，并对生命值、知识值、人际关系、金钱和学分等属性产生影响。程序加入随机事件机制、难度选择系统、多结局系统以及输入校验机制。游戏最终根据玩家属性综合判断普通毕业、完美毕业、学业警告和毕业失败等不同结果。

评分项	评分标准	自 评 分 数	教 师 确 认
游 戏 界 面 (10 分)	10 分：界面完整，各元素清晰，有游戏截图 7-9 分：界面基本完整，部分元素需改进 4-6 分：界面简陋但可用 0-3 分：界面不完整或无法使用	9	

游 戏 逻 辑 (10 分)	10 分：完整实现选择系统和属性影响机制 7-9 分：基本实现核心功能，有小缺陷 4-6 分：部分功能实现，存在明显缺陷 0-3 分：核心功能缺失或无法运行	9	
-------------------	--	---	--

第二部分 技术实现（30 分）

代码质量分析（10）：

本程序采用模块化设计，将不同功能划分为多个独立函数。输入处理、状态显示、地点选择、事件执行和结局判断分别由不同函数负责，使程序结构更加清晰。程序使用统一变量管理玩家状态，包括生命值、知识值、人际关系、金钱和学分等属性。变量命名采用英文形式，能够较清楚地表达变量含义，提高代码可读性。

输入部分加入输入标准化与输入校验机制，可处理大小写、空输入和非法输入问题，提高程序稳定性。主函数 `main()` 负责整体流程控制，包括玩家输入、难度选择、十二周循环、状态更新和最终结果输出，整体逻辑完整。

```
def normalize_input(text):
    return text.strip().lower().replace(" ", "")

def make_bar(value, max_value=100, length=20):
    """Create a simple text progress bar."""
    if value < 0:
        value = 0
    if value > max_value:
        value = max_value
```

图 2.1 输入标准化与文本进度条实现代码

该部分代码用于处理用户输入格式，并通过文本进度条实现玩家状态的可视化显示。

```
def choose_location():
    print("\nWhere do you want to go this week?")
    print("1. Library")
    print("2. Cafeteria")
    print("3. Club Room")
    print("4. Professor Office")
    print("5. Dorm")
```

图 2.2 地点选择与交互逻辑代码

程序通过菜单选项与输入映射机制实现地点选择，并根据玩家输入执行对应事件。

代码对应的 Python 知识点和 AI 使用说明（10 分）：

无 AI 辅助

- 变量：使用变量保存和更新玩家的生命值、知识值、人际关系、金钱和学分等状态信息。
- 条件判断：使用 `if`、`elif`、`else` 实现事件判断、输入判断与结局判断。
- 循环结构：使用 `for` 循环模拟十二周游戏流程，使用 `while` 循环实现输入校验。
- 输入输出：使用 `print()` 与 `input()` 实现玩家交互与状态显示。
- 随机事件：使用 `random.choice()` 与 `random.randint()` 实现随机事件系统。
- 自定义函数：使用 `def` 定义不同功能模块，例如状态显示、地点选择与结局判断。
- 字符串处理：使用 `strip()`、`lower()` 与 `replace()` 实现输入标准化处理。
- 字典映射：使用 `dictionary` 建立输入与地点函数之间的映射关系，提高程序结构清晰度。
- 文本可视化：通过 `make_bar()` 与 `show_stat_bar()` 实现文本 Progress Bar 状态显示。

技术创新和源码注释及高亮（10 分）：

```
# 导入 random 库，用于生成随机事件
import random

# 输入标准化函数：
# 去除首尾空格，统一小写，并删除中间空格
def normalize_input(text):
    return text.strip().lower().replace(" ", "")

# 文本进度条函数：
# 将玩家状态数值转换成可视化进度条
def make_bar(value, max_value=100, length=20):
    if value < 0:
        value = 0
    if value > max_value:
        value = max_value

    filled = int(value / max_value * length)
    empty = length - filled
    return "[" + "#" * filled + "-" * empty + "]"

# 状态显示函数：
# 输出生命值、知识值、人际关系、金钱和学分
```

```

def show_stat_bar(name, value, max_value=100):
    bar = make_bar(value, max_value)
    print(f"{name:<14} {bar} {value}/{max_value}")

# 当前状态提示函数:
# 根据玩家属性给出简短建议
def get_status_label hp, knowledge, relation, money, credit):
    if hp <= 25:
        return "Danger: Your health is very low. You should rest soon."
    elif money <= 10:
        return "Warning: You are almost out of money. A part-time job may be
needed."
    elif credit >= 12 and knowledge >= 70:
        return "Good: You already meet the basic graduation conditions."
    elif knowledge < 50:
        return "Study Needed: Your knowledge level is still low."
    elif relation < 30:
        return "Social Needed: Your relationship score is low."
    else:
        return "Stable: Your campus life is balanced."

# 难度选择函数:
# 不同难度对应不同初始属性
def choose_difficulty():
    print("\nChoose difficulty:")
    print("1. Easy")
    print("2. Normal")
    print("3. Hard")

    while True:
        choice = normalize_input(input("Enter difficulty: "))

        if choice == "":
            print("Input cannot be empty. Please choose a difficulty.")
            continue

        if choice in ["1", "easy"]:
            return "Easy", 100, 35, 25, 70
        elif choice in ["2", "normal"]:
            return "Normal", 80, 30, 20, 50
        elif choice in ["3", "hard"]:
            return "Hard", 70, 25, 15, 35
        else:

```

```

        print("Invalid input. Please enter 1, 2, 3 or difficulty name.")

# 地点选择函数:
# 使用字典将输入内容映射到对应事件函数
def choose_location():
    mapping = {
        "1": library_event,
        "library": library_event,

        "2": cafeteria_event,
        "cafeteria": cafeteria_event,

        "3": club_event,
        "club": club_event,
        "clubroom": club_event,

        "4": professor_event,
        "professor": professor_event,
        "office": professor_event,
        "professoroffice": professor_event,

        "5": dorm_event,
        "dorm": dorm_event
    }

    while True:
        choice = normalize_input(input("Enter your choice: "))

        if choice == "":
            print("Input cannot be empty. Please choose a location.")
            continue

        if choice in mapping:
            return mapping[choice]()

        print("Invalid choice. Please enter a number or location name.")

```

技术创新说明:

- 本项目通过 `normalize_input()` 实现输入标准化, 支持数字、英文、大小写混合等多种输入方式, 提高了交互容错率。
- 通过 `make_bar()` 和 `show_stat_bar()` 实现文本进度条, 将玩家状态从单纯数字转换为可视化显示, 增强了控制台界面的直观性。

- 通过 while True 循环实现输入校验，避免空输入或非法输入导致游戏流程异常。
- 通过 dictionary 映射机制将玩家输入与对应事件函数连接，使地点选择模块结构更加清晰。
- 通过难度选择、状态提示和随机事件机制，增强了游戏的策略性与可玩性。

评分项	评分标准	自评分数	教师确认
代码质量 (10分)	10分 : 代码结构清晰，命名规范，有注释 7-9分 : 代码基本规范，结构较清晰 4-6分 : 代码可读性一般，结构需改进 0-3分 : 代码混乱，难以理解	9	
AI 辅助使用 (10分)	10分 : AI 使用率<30% ，并有明确说明使用方式 7-9分 : AI 使用率<30%，说明不够详细 4-6分 : AI 使用率接近 30%，缺乏说明 0-3分 : AI 使用率 >30%或无说明	9	
技术创新 (10分)	10分 : 使用了高级技术或创新解决方案 7-9分 : 有一定技术亮点 4-6分 : 使用基本技术实现功能 0-3分 : 技术实现简单，缺乏亮点	8	

第三部分：学习成果（20 分）

本项目加深了我对 Python 基础语法和程序结构的理解。项目中使用了变量、条件判断、循环、函数、字符串处理、字典映射和随机数等知识点，并将多个知识点结合完成完整程序。代码采用模块化设计，提高了程序结构清晰度。输入部分加入输入校验机制，增强了程序稳定性。项目中还实现了状态管理系统、多结局系统和随机事件机制。程序调试过程中，我不断调整属性平衡、输入逻辑和游戏流程，提高了问题分析与调试能力。本次实践使我更加熟悉 Python 中的函数结构、流程控制和交互式程序设计，也提高了独立完成小型项目的能力。

评分项	评分标准	自评分数	教师确认
能力收获（ 10 分）	10分 : 详细描述 3 项以上具体能力提升 7-9分 : 描述 2-3 项能力提升 4-6分 : 简单描述 1-2 项能力提升 0-3分 : 未描述或描述不具体	9	

时间投入与产出（10分）	10分： 详细记录时间投入，产出比高 7-9分： 有时间记录，产出比合理 4-6分： 时间记录不详细，产出一般 0-3分： 无时间记录或产出比低	8	
--------------	--	---	--

第五部分：教学反馈（10分）

分项	评分标准	自评分数	教师确认
反馈质量（10分）	10分： 提供具体、有建设性的反馈，至少3条 7-9分： 提供较有价值的反馈，2-3条 4-6分： 提供一般性反馈 0-3分： 反馈模糊或缺失	10	